

AWS Architecture & Cloud Services

A comprehensive guide — Glue · Redshift · AppFlow · Athena · Route 53 · CloudFront · ALB · Disaster Recovery

1. AWS Glue — Use Cases

What is AWS Glue?

AWS Glue is a fully managed, serverless ETL (Extract, Transform, Load) service. It handles data preparation and integration so you can move and transform data between AWS services without managing infrastructure.

Key Use Cases

1. ETL Pipelines

Extract data from S3, RDS, or APIs, transform it (clean, filter, enrich), and load it into a data warehouse or data lake. No server management needed.

2. Data Cataloguing

Automatically crawl your S3 buckets or databases and build a metadata catalogue — so other services like Athena or Redshift know what data exists and its schema.

3. Schema Discovery & Evolution

When your source data structure changes (new columns, different formats), Glue detects and adapts automatically without breaking your pipeline.

4. Data Lake Preparation

Convert raw files (CSV, JSON, XML) into optimised formats like Parquet or ORC before storing in S3 — making downstream queries much faster and cheaper.

5. Event-Driven Ingestion

Trigger Glue jobs automatically when new files land in S3 — useful for real-time log processing or frequent batch feeds.

2. Amazon Redshift — Use Cases

What is Redshift?

Amazon Redshift is a fully managed, petabyte-scale cloud data warehouse. It uses columnar storage and massively parallel processing (MPP) for fast analytical queries on large datasets.

Key Use Cases

1. Business Intelligence & Reporting

Connect BI tools like Tableau, Power BI, or QuickSight directly to Redshift for dashboards and executive reporting on large datasets.

2. Historical & Trend Analysis

Store years of transactional data and run complex analytical queries — e.g. sales trends, customer churn patterns, revenue forecasting.

3. Data Warehousing

Central repository that consolidates data from multiple sources (CRM, ERP, marketing tools) into one place for unified analytics.

4. Ad-hoc Querying at Scale

Run complex SQL queries across billions of rows in seconds using Redshift's columnar storage and massively parallel processing.

5. Redshift Spectrum

Query data directly in S3 without loading it into Redshift — great for cold or archival data you don't want to pay to store in the warehouse.

How Glue + Redshift Work Together

Raw Data (S3 / RDS / APIs) | AWS Glue (transform, clean, convert to Parquet) | Redshift (load & store for analytics) | BI Tools (Tableau / QuickSight)

Glue handles the messy ingestion and transformation. Redshift handles the fast analytical querying. Together they form the backbone of most modern data warehouse architectures on AWS.

3. AWS AppFlow — Use Cases

What Problem Does AppFlow Solve?

Glue is great for data sitting in AWS (S3, RDS, DynamoDB). But what if your data lives in SaaS tools like Salesforce, SAP, ServiceNow, Marketo, Google Analytics, or Zendesk? Writing custom API connectors for each of these in Glue is painful. AppFlow does this out of the box — no code needed.

Where AppFlow Fits

SaaS Apps AWS (Salesforce, SAP, ServiceNow, Marketo) -> AppFlow -> S3 / Redshift | Glue (transform if needed) | Redshift | BI Tools

Key Use Cases

- Salesforce to Redshift — Pull CRM data (leads, opportunities, accounts) directly into Redshift for sales analytics without writing a single API call.
- Marketo / Google Analytics to S3 — Ingest marketing campaign data into your data lake for attribution analysis.
- SAP to S3 — Extract ERP data (orders, inventory, finance) and land it in S3 for Glue to transform.
- Event-Driven Sync — Trigger a flow when a record changes in Salesforce — near real-time data sync without polling APIs manually.
- Bi-directional Flow — AppFlow can also push data back into SaaS apps, e.g. send enriched customer segments from Redshift back into Salesforce.

AppFlow vs Glue

Scenario	Use
SaaS to AWS data ingestion	AppFlow
Complex data transformation	Glue
S3 / RDS / DynamoDB pipelines	Glue
No-code, scheduled SaaS sync	AppFlow
Custom API sources	Glue (with custom connectors)
Real-time SaaS record changes	AppFlow

4. Amazon Athena — Use Cases

What is Athena?

Athena is serverless SQL — point it at your S3 data, write a query, get results. No cluster, no loading, no infrastructure. You pay only per TB of data scanned.

Where Athena Fits

SaaS Apps -> AppFlow --| |--> S3 (Raw Data Lake) DB / APIs -> Glue --| | | Athena Glue (ad-hoc/logs) (transform) | Redshift

Key Use Cases

- Ad-hoc Exploration on Raw Data — Before cleaning data, analysts can run SQL instantly on raw S3 data without waiting for a pipeline.
- Log Analysis — Query CloudTrail, ALB access logs, VPC flow logs, CloudFront logs stored in S3. No need to load billions of lines into Redshift.

- **Cost-Sensitive Querying** — For infrequent queries on cold/archival data, Athena is far cheaper than keeping a Redshift cluster running 24/7.
- **Data Lake Querying** — Query Parquet/ORC files partitioned by date. Athena queries only the relevant partitions — fast and cheap.
- **Glue Catalogue Integration** — Athena uses the Glue Data Catalogue as its metadata store. Whatever Glue has crawled, Athena can query immediately.
- **Federated Queries** — Athena can query S3, RDS, DynamoDB, and on-premise databases in a single SQL statement.

Athena vs Redshift

Scenario	Use
Frequent, complex BI queries	Redshift
Ad-hoc, one-off queries on S3	Athena
Cold / archival data	Athena
Dashboards & reporting tools	Redshift
Log analysis	Athena
High concurrency analytics	Redshift

One Line Summary for Each Service

Service	One Line
AppFlow	Pull data from SaaS apps into AWS
Glue	Transform and prepare data
Athena	Query raw data in S3 with SQL
Redshift	Fast analytics on structured warehouse data

5. Disaster Recovery — RTO & RPO

RPO — Recovery Point Objective

"How much data loss can we tolerate?"

It defines the maximum age of data you can recover from. In other words — how far back in time are you willing to go?

Example: If your RPO is 4 hours and a disaster strikes at 2 PM, you must be able to restore data from at least 10 AM. Any data between 10 AM and 2 PM may be lost — and that is acceptable.

RPO drives your backup frequency. RPO of 4 hours = backup every 4 hours. RPO of 0 = continuous replication, no data loss tolerated.

RTO — Recovery Time Objective

"How fast must we be back online?"

It defines the maximum downtime your business can tolerate after a disaster.

Example: If your RTO is 2 hours, your systems must be fully restored and operational within 2 hours of the disaster.

RTO drives your recovery infrastructure. RTO of 24 hours = simple backups are fine. RTO of 15 minutes = you need hot standby systems running at all times.

The Cost vs Risk Trade-off

RPO / RTO	Strategy	Cost
Hours / Days	Simple backups & restore	Low
Minutes / Hours	Warm standby	Medium
Near zero / Minutes	Hot standby / Multi-region active-active	High

In AWS Context

Goal	AWS Approach
Low RPO for database	RDS automated backups, continuous replication
Low RPO for S3	S3 Cross-Region Replication
Low RTO	Multi-AZ deployments, Route 53 failover
Near-zero both	Active-Active multi-region with Aurora Global
High RTO/RPO acceptable	Backup to S3 Glacier, restore when needed

6. Route 53 — Use Cases

What is Route 53?

Think of it as the internet's phone book. When you type `www.myapp.com`, something needs to translate that name into an actual server IP address. That is Route 53's core job — DNS (Domain Name System). But it does a lot more than just that.

Use Case 1 — Basic DNS

User types `myapp.com` | Route 53 | Returns `54.23.11.5` | Browser connects to server

Use Case 2 — Latency Based Routing

Route 53 detects where the user is and routes them to the closest server — reducing latency.

User in India -> Route 53 -> Asia server User in UK -> Route 53 -> Europe server User in USA -> Route 53 -> US server

Use Case 3 — Failover & Disaster Recovery

Route 53 detects primary server failure via health checks and automatically redirects traffic to your standby server.

Normal: User -> Route 53 -> Primary Server OK Failure: User -> Route 53 -> Detects failure -> Backup Server OK

Use Case 4 — Blue/Green Deployments

Shift traffic gradually between old (Blue) and new (Green) versions using Weighted Routing.

Week 1: 90% -> Blue, 10% -> Green Week 2: 50% -> Blue, 50% -> Green Week 3: 0% -> Blue, 100% -> Green

Use Case 5 — Internal DNS for AWS Resources

Route 53 Private Hosted Zones let services call each other by name internally instead of hardcoding IPs.

All Routing Types

Routing Type	Use Case
Simple	One domain to one server
Latency	Route to nearest/fastest server
Failover	Primary down — switch to backup
Weighted	Split traffic by percentage
Geolocation	Route by user country/region
Geoproximity	Route by physical distance
Multi-value	Return multiple IPs, client picks

7. Production Grade Patterns — R53, ALB, API Gateway, CloudFront

R53 is NOT a Typical Load Balancer

Capability	Route 53	ALB
Level	DNS level	Network / App level
Sees	Domain names	Actual HTTP requests
Routes based on	DNS queries	Request headers, paths, cookies
Health check speed	Slow (30s intervals)	Fast (5-30s, instance level)
Sticky sessions	No	Yes
SSL termination	No	Yes
Instance level routing	No	Yes

Pattern 1 — R53 → ALB → EC2 / ECS

For traditional web apps and containerised microservices. Use when running containers on ECS/EKS, traditional web servers, WebSocket support needed, or high throughput requirements.

```
User -> Route 53 -> ALB -> EC2 / ECS (multiple instances)
```

Pattern 2 — R53 → API Gateway → Lambda

For fully serverless APIs. Use when traffic is spiky, you need built-in auth/throttling, or for rapid API development.

```
User -> Route 53 -> API Gateway -> Lambda (auto scales)
```

Pattern 3 — Avoid: R53 → ALB → API Gateway

Adding ALB in front of API Gateway adds latency and cost with no real benefit. Pick one.

Pattern 4 — Production Grade: R53 → CloudFront → ALB / API Gateway

This is the pattern most large production systems use. CloudFront provides CDN, WAF, SSL termination, and DDoS protection at the edge.

```
User | Route 53 (myapp.com) | CloudFront (CDN + WAF + SSL) | | ALB API Gateway  
(web/containers) (APIs/Lambda) | | EC2 / ECS Lambda | RDS / ElastiCache
```

Why CloudFront Does More Than CDN

CloudFront uses Origins and Behaviours to act as a smart reverse proxy:

Path Pattern	Origin	Cache?
/api/*	ALB / API Gateway	No (dynamic)
/images/*	S3	Yes (long TTL)
/* (default)	S3	Yes (cached)

CloudFront checks the path, forwards /api/* to the backend with no caching, and serves static files from S3 at the edge. One entry point, intelligent routing behind it.

How R53 Points to AWS Resources — Alias Records

```
myapp.com -> Alias -> CloudFront URL (abc123.cloudfront.net) api.myapp.com -> Alias ->  
API Gateway URL (xyz.execute-api.amazonaws.com)
```

You never hardcode an IP. AWS manages the IPs behind ALB and API Gateway dynamically. Route 53 Alias records handle that automatically.

8. Health Checks — R53 vs ALB

Route 53 Health Check

Route 53 sends an HTTP/HTTPS request to your endpoint every 30 seconds:

```
Route 53 -> GET https://myapp.com/health | Returns 200 OK? -> Healthy No response? ->  
Unhealthy Returns 500? -> Unhealthy
```

What Each Layer Health Checks

Layer	What it checks
Route 53	Is the endpoint reachable at all? (DNS level)
CloudFront	Is the origin responding?
ALB	Are individual EC2/ECS instances healthy? (instance level)

When ALB Health Checks Are Essential

ALB does something unique — instance level health checks. If one EC2 instance crashes, ALB removes it from rotation immediately. R53 and CloudFront cannot do this granularity.

When R53 Health Checks Are Sufficient

Scenario	Need ALB health checks?
R53 -> CloudFront -> S3	No — S3 is managed, no instances
R53 -> CloudFront -> API Gateway -> Lambda	No — serverless
R53 -> CloudFront -> ALB -> EC2 / ECS	Yes — instance level needed

9. Multi-Region Architecture — Geo Routing + Failover

Full Regional Stack

Each region is a complete independent stack — not just a redirect URL.

```
Route 53 | | Primary (us-east-1) Secondary (ap-south-1) CloudFront CloudFront | | ALB
(us-east-1) ALB (ap-south-1) | | ECS / EC2 ECS / EC2 | | RDS Primary RDS Read Replica
```

Scenario 1 — Geolocation Routing (Normal)

Each region handles its own users all the time. R53 acts as a smart postman.

```
Indian user -> R53 -> ap-south-1 ALB -> India servers US user -> R53 -> us-east-1 ALB ->
US servers
```

Scenario 2 — Failover (Disaster Recovery)

When primary region fails, R53 health check detects it after 3 consecutive failures and flips DNS to secondary region.

```
Normal: All users -> R53 -> Primary ALB (us-east-1) OK Disaster: Primary ALB fails health
check | R53 flips DNS to Secondary ALB (ap-south-1) | All users now routed to secondary
region
```

Combined: Geo Routing + Failover

```
Indian user (normal) -> R53 -> ap-south-1 OK Indian user (ap-south-1 down) -> R53 ->
us-east-1 OK (failover) US user (normal) -> R53 -> us-east-1 OK US user (us-east-1 down)
-> R53 -> ap-south-1 OK (failover)
```

Database Strategy During Failover

Strategy	How
RDS Read Replica	Promote replica to primary during failover
Aurora Global	Sub-second replication, automatic failover
DynamoDB Global Tables	Multi-region active-active, no failover needed

Cost vs RTO Trade-off

Approach	Secondary Region	Cost	RTO
Active-Active	Fully running, takes live traffic	High	Seconds
Warm Standby	Running but scaled down	Medium	Minutes
Pilot Light	Only DB running	Low	30-60 mins
Backup & Restore	Nothing running	Cheapest	Hours

Key Takeaways

- R53 is a DNS-level traffic director — not a load balancer. It answers 'which region?' not 'which server?'
- ALB answers 'which instance inside the region?' — essential for EC2/ECS workloads.
- CloudFront is more than a CDN — it's a global reverse proxy with WAF, SSL termination, and path-based routing.
- AppFlow fills the SaaS connectivity gap that Glue doesn't cover natively.
- Athena and Redshift complement each other — Athena for ad-hoc S3 queries, Redshift for production BI.
- RPO drives backup frequency. RTO drives infrastructure investment.
- Every region in a multi-region setup needs a complete independent stack — not just a redirect.

Generated from conversation on AWS architecture and cloud services best practices.